

# Coding Dojo: an Object Oriented Programming Kata

---

In this kata, we will practice principles of object oriented programming (OOP). Specifically, we will address topics like instantiation, inheritance, composition, abstract classes and interfaces, encapsulation, immutability, polymorphism, etc.

Here we return to our TDD principles. So for all problems, before you write some implementation, write a test first!

## Object and Class

---

First some definitions: an object is something that has properties and can perform actions. A class is a blueprint for an object. From a class you can create ("instantiate") multiple objects of the same type, but with possibly different properties.

We want to model computational grids for numerical simulations. A 1D grid has coordinates (points in space) and a spacing. Write a class `Grid1D` that captures this notion. `Grid1D` objects can print themselves in a formatted way.

Write tests that assert that `Grid1D` objects

- can be instantiated with different properties (start, end, number of points)
- raise a `ValueError` when instantiated with invalid arguments (e.g., `n_points < 2`)
- convert into a properly formatted way when converted to a string
- are equal if and only if their coordinates agree (use `numpy.allclose`)
- have a property `coords` that returns a numpy array of grid points
- have a property `spacing` that returns the grid spacing (dx)

## Composition

---

Composition is a fundamental OOP principle where objects are built by combining other objects. Instead of inheritance ("is-a" relationship), composition models a "has-a" relationship. For example, a car "has-a" engine rather than "is-an" engine.

Then write a class `ScalarField1D` class that captures the notion of a scalar field on a one-dimensional domain (i.e. grid). So a field has values and has a grid.

Write tests that assert that `ScalarField1D` objects

- can be instantiated with a `Grid1D` object and an array of values
- raise a `ValueError` when the number of values doesn't match the grid size
- have a property `grid` that returns the associated `Grid1D` object
- have a property `values` that returns the numpy array of field values
- convert into a properly formatted string representation
- are equal if and only if their grids and values agree (use `numpy.allclose` for values)

## Arithmetic with Dunder Methods

---

Fields can be added, subtracted (when they shared a grid) and multiplied by scalars. Implement the corresponding "dunder" methods. Tests first!

## Inheritance

---

Inheritance is another key OOP principle where a class (subclass/child) can inherit properties and methods from another class (superclass/parent). This enables code reuse and establishes an "is-a" relationship.

Create a hierarchy of grid classes: `Grid` as the abstract base class, `Grid2D` as an abstract 2D grid class, and concrete implementations `CartesianGrid2D` and `PolarGrid2D`. Both `Grid1D` and `Grid2D` should share common functionality from `Grid`.

Write tests that assert:

- `Grid` is an abstract base class that cannot be instantiated directly
- `Grid2D` is also abstract and cannot be instantiated directly
- Both `Grid1D` and `Grid2D` inherit from `Grid`
- `CartesianGrid2D` can be instantiated with start/end coordinates and number of points in both x and y directions
- `CartesianGrid2D` has a property `mesh` that returns 2D coordinate arrays (use `numpy.meshgrid`)
- `PolarGrid2D` can be instantiated with radial (r) and angular (theta) ranges and number of points
- `PolarGrid2D` has a property `mesh` that returns 2D arrays for the r and theta coordinates
- Common methods (like `__str__` or validation) are inherited from the base `Grid` class

Extend your field classes to support 2D grids as well. Create a `ScalarField2D` class that works with 2D grids (`CartesianGrid2D` or `PolarGrid2D`).

Write tests that assert:

- `ScalarField2D` can be instantiated with a 2D grid and a 2D array of values
- The shape of the values array must match the grid dimensions (raises `ValueError` otherwise)
- `ScalarField2D` supports the same arithmetic operations as `ScalarField1D` (addition, subtraction, scalar multiplication)
- Consider creating a base `ScalarField` class that both `ScalarField1D` and `ScalarField2D` inherit from
- Fields can only be combined arithmetically if they have compatible grids

## Instance variables vs. class variables

---

Instance variables are unique to each object, while class variables are shared across all instances of a class.

Add a class variable `grid_count` to the `Grid` base class that tracks the total number of grid instances created (across all grid types). Also add a class variable

`dimension` to each concrete grid class ( `Grid1D`, `CartesianGrid2D`, `PolarGrid2D` ) that stores the dimensionality.

Write tests that assert:

- `Grid.grid_count` increments each time any grid object is created
- `Grid1D.dimension` equals 1 and is shared across all `Grid1D` instances
- `CartesianGrid2D.dimension` and `PolarGrid2D.dimension` both equal 2
- Modifying an instance variable (like `values` in a `ScalarField1D` ) only affects that specific instance
- Modifying a class variable through the class affects all instances
- Each instance can still override a class variable with an instance variable of the same name

## Encapsulation light (Python-Style), Computed Properties

---

Encapsulation is about controlling access to an object's internal state. Python uses conventions (like `_private` attributes) rather than strict enforcement. Computed properties allow you to define methods that can be accessed like attributes using the `@property` decorator.

Modify your grid classes to use encapsulation:

Write tests that assert:

- Grid coordinates are stored in a "private" attribute (e.g., `_coords` )
- Coordinates are accessed through a public `coords` property
- Attempting to set the `coords` property raises an `AttributeError` (read-only property)
- The `spacing` is a computed property (calculated on-the-fly, not stored)
- `ScalarField1D` has a computed property `max_value` that returns the maximum field value
- `ScalarField1D` has a computed property `min_value` that returns the minimum field value

- Grid properties validate data when set through setters (if you choose to make them writable)

## Polymorphism & Duck Typing

---

Polymorphism allows objects of different types to be used interchangeably if they share a common interface. Duck typing is Python's approach: "If it walks like a duck and quacks like a duck, it's a duck." The focus is on what an object can do, not what it is.

Create a `GradientOperator` class that can compute gradients of scalar fields.

Use proper encapsulation and the factory pattern: - Grid classes provide a

`compute_gradient(values)` method (grids know how to compute gradients) -

Field classes provide a `with_values(new_values)` factory method (fields know how to create new instances) - The operator simply coordinates: delegates computation to grid, uses field's factory for result

This way, the operator has no knowledge of specific grid or field types!

Write tests that assert:

- Each grid class has a `compute_gradient(values)` method
- Each field class has a `with_values(new_values)` factory method
- `GradientOperator` can be instantiated and applied to `ScalarField1D` objects
- For 1D fields, gradients are computed using the grid spacing
- `GradientOperator` works with 2D scalar fields ( `ScalarField2D` ) without knowing the grid or field type
- The operator is completely decoupled from concrete types (no isinstance checks in `GradientOperator.apply`)